SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 12 October 2026

Abstract

Games, films and robot training need 3D worlds that are complete, editable and owned by the people who make them. World models and one-shot 3D generators can produce a convincing place from a sentence or a picture, but they give frames that no engine can open, or one fused scene without separate objects or clear rights. Studios have always built worlds in stages, from concept art and layout to modelling, surfacing, sound and review. We present SCORE, a world generation framework that keeps these stages and automates large parts of them with open vision foundation models, procedural tools and coding agents, while the creator chooses and reviews. Because every stage's output is kept, the creator can rerun or fix any single stage, and because every model allows commercial use, the creator stays the author of the result, holding its full commercial rights. We are building SCORE's first world, the game world of 2099, only 3 of whose 18 places passed its completion gate when it was last measured, and plan three further worlds of different kinds. Our aim is to show that checkable stages, with open models for shape and closed, paid models for pictures and surfaces, can build worlds that a creator owns and keeps editing. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: teaser figure. This figure will show four worlds made with SCORE: the game world of 2099, a stylised underwater world, a simulation world for robots, and a third-person fantasy world. Each will appear as its chosen concept beside the finished world in its engine. It waits until the four worlds are finished and accepted by their creators.

1 Introduction

Interactive 3D worlds are the raw material of games, of film and of embodied AI, where agents are trained and tested in simulators [1]. Building one is still expensive: every object has to be modelled, given surfaces, placed, given collision and physical properties, lit and given sound, all in one consistent style, and the world has to stay editable while the project changes. Generative models promise to take over much of this work, but the people who would use them are wary of it: in GDC's 2026 survey, 36 percent of game workers used generative AI at work, and 52 percent thought it was hurting the industry [2]. A tool for building worlds therefore has to leave the result in the creator's hands.

Two lines of research generate the world itself. The first, learned world models, generates it as frames: Genie learns an interactive environment from video [3], and Genie 3 renders a world that a person can move through in real time, for a few minutes of continuous interaction [4]. Nothing that these models produce can be opened in a game engine or an editor. The second line, 3D world generators, produces geometry instead. WorldGen [5] and HY-World 2.0 [6] turn text or a picture

into a traversable scene or a single splat or mesh world, but most such outputs are "static monolithic assets with limited editability and physical interaction" [7], and WorldAct and WorldSculpt recover separate objects from them after the fact [7], [8]. In both lines the world is generated in one shot. When part of the result is wrong, the creator can regenerate the whole world or repair it by hand, but cannot see which step caused the fault, and cannot change that step alone.

A third line of work keeps the world explicit. Infinigen and Infinigen Indoors build every asset from procedural rules [9], [10], which gives real geometry and full control, but only over the content that the rules describe. Holodeck has a language model write layout constraints that a solver fills with assets from a library [1], and SceneCraft, recursive code world models, WorldClaw and AutoUE let coding agents write scenes, or whole games, as programs [11], [12], [13], [14]. LEGO-Anything shows that coding agents rebuilding a scene from a single picture judge their own geometry unreliably, and that grounded tools, rather than further training, close much of that gap [15]. All of these systems, however, either rebuild a scene that already exists or draw their objects from a fixed library of assets.

Meanwhile, each of the parts that a world needs can now be read out of pictures by an open model. Vision foundation models [16] can turn a picture into a 3D shape aligned with its pixels [17], into a mask for each named object [18], into depth in metres [19], [20], into the parts of an object [21] or into a human body [22]; related models turn a sentence into motion [23] or into sound [24]. Most of these models pass on pictures, masks, depth maps or meshes rather than their internal features, so a coding agent can connect them to one another as it would connect tools. Chorus instead distils several 2D foundation models into one encoder of an existing 3D scene [25]; SCORE builds the scene and joins its models only through what they output, so that any of them can be replaced. What none of them provides is one consistent style across objects that different models have made. The field names this as an open problem [26], [27], and Hunyuan3D Studio addresses it only by restyling the input picture [28].

Studios have always built worlds in stages: concept art, layout, asset lists, modelling, surfacing, sound and review. Each stage has an output that a person can check before the next one starts. SCORE keeps these stages and automates large parts of them with the models above, with procedural tools and with coding agents, while the creator chooses the style, the concepts and the storyboards up front and reviews each stage. Its name states what it aims for. The worlds it makes are **complete**: every object, surface, light and sound exists wherever the player looks, and what the core mechanics need, such as collision and the doors that open, is made when the world is built. They are **owned**: the creator stays the author of the result, holding its full commercial rights, because every model in the framework allows commercial use of its output. They are **responsive**: every object has real collision, mass and material, so it behaves physically in any engine or simulator, under physics that each world may set for itself, such as the low gravity of the Moon. And they can be any kind of **environment**, real or made up. SCORE takes procedural tools, part splitters and world generators in as building blocks wherever their licences allow. A place is one OpenUSD stage, which opens in the tools that artists and engineers already use.

To our knowledge, no other openly available world generation work joins vision foundation models into engine-ready scenes that are built from reference pictures, steered by a person's choices at each step, and made only with models that allow commercial use of their output.

This paper makes three contributions:

- **The SCORE framework:** staged, retraceable generation from a creator's choices to one canonical scene, which consists of a generated base layer and the creator's own edit layers

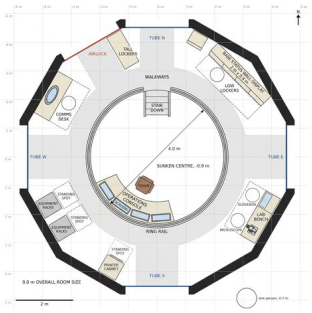
(Section 2).

- **The mechanisms that keep its worlds coherent and make faults cheap to fix:** deterministic checks before every stage that costs money, a parts check that decides how each object is built so that every real-world object becomes one model, a shared library of rule-based surfaces that keeps shape and surface apart, and review tools for the creator (Section 2).
- **Evidence from four worlds of different kinds:** the time, the cost and the faults caught before spending at every stage, and an evaluation on a shared benchmark (Sections 3 and 4).

2 Methods



(a) concept, picked by the creator



(b) dimensioned plan

scene inventory (excerpt)

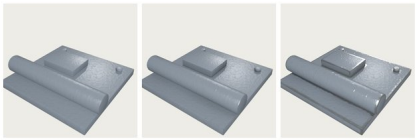
id	anchor	size	count
console	floor	2.6x0.6x1.2 m	x1
labbench	floor	2.0x0.7x1.5 m	x1
chair	floor	0.6x0.6x0.9 m	x1
talllocker	floor	1.2x0.55x2.0 m	x1
rack	floor	0.6x0.6x1.9 m	x2
comms	floor	1.5x0.7x1.2 m	x1
toolboard	wall	1.2x0.08x0.8 m	x1

... 74 rows in all

(c) inventory



(d) close-ups



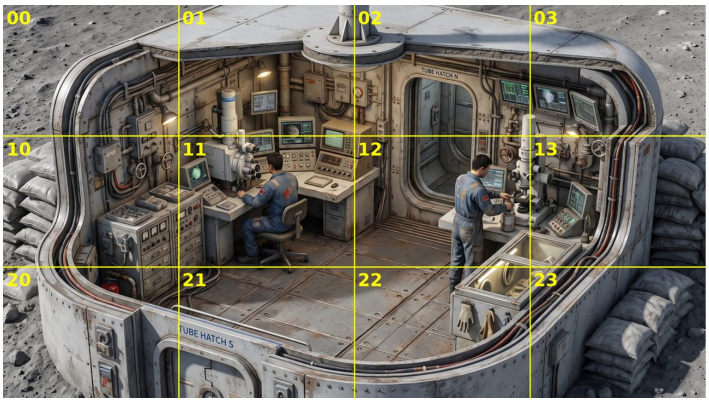
(e) library surface, three wear levels



(f) assembled room in the engine

Figure 1: The stages on one place, the central hub of the game 2099. (a) The concept, drawn by a picture model from the creator's reference pictures and chosen by him. (b) The dimensioned plan in metres. (c) An excerpt of the scene inventory, which has 74 rows. (d) Close-ups of two objects. (e) One library surface at three wear settings. (f) The assembled room in the game engine.

Figure 1 shows the stages on one place. The creator chooses a concept (a), and a coding agent turns it into a dimensioned plan (b) and a scene inventory (c). An open picture model draws a close-up of every object in the inventory, and a closed one redraws those that fail a shape check (d). Each object is then built in code or generated in 3D, its parts are painted from the shared library of rule-based surfaces (e), and the place is assembled in an engine (f). The output of every stage is kept, so the creator can trace a weak result to the stage that caused it, rerun or fix that stage, and regenerate the stages after it while keeping their own edits. Every stage that loads a model runs on rented cards that are deleted after each batch, or as a job in a pinned container image on any Kubernetes cluster, a path not yet used for world 1.



each element of the concept -> an inventory row

tile 11 01 13: research microscope
  -> microscope
tile 12 22: glovebox cabinet
  -> glovebox_cabinet
tile 10 12 13: valves on the gas lines
  -> pipe_valve
tile 10: fire extinguisher
  -> fire_extinguisher

tile 02 03: three wall monitors
  -> wall_monitor
tile 21: coil of cables on the floor
  -> dropped: loose cable on a walkway floor
tile 12: petri dish, small tools
  -> dropped: under 10 cm, unreadable from...
tile 11 12: two crew at the desks
  -> dropped: people are the crew's, not the room's
47 elements; 7 dropped with a reason

Figure 2: The inventory of the lab on the Moon base. Left: the concept, cut into twelve tiles. Right: how visible elements map to inventory rows or are dropped with a written reason; of the concept's 47 elements, 7 were dropped.

2.1 Inventory as a partition of the concept

The inventory decides what exists in a place. A coding agent cuts the concept into tiles and lists every visible element, and each element either becomes a row (with its kind, anchor, size and count) or is dropped with a written reason (Figure 2). Nothing is made or placed that is not a row. After the build, the place is rendered from the concept's own camera and compared with the concept; a place that is clearly sparser than its concept fails this concept-density check.

2.2 Close-ups: the open model first

Every object is built from a close-up: a picture of the object alone on a plain background, drawn from its crop of the concept. An open model, Qwen-Image-Edit-2511 [29], draws every close-up first, and a shape check decides, without a person, whether it may go on to the 3D step: it measures the picture's border, framing and outline, and an open vision-language model, Qwen3.8-27B [30], compares it with the crop by a majority of three answers to set questions (Appendix D). Only a close-up that fails is redrawn by Nano Banana Pro [31], first from the crop alone and then with the whole concept beside it, and its pictures pass the same check. The check leans towards failing, because a wrong picture that passes costs a bad 3D model, while a good one that fails costs only a picture from the closed model (Appendix E).

2.3 Code or generation: the parts check

Picture-to-3D models build chunky solids well, but flat panels, thin beams and openings badly, while code builders are exact and straight but show only the parts that the coding agent wrote. For each kind of object, the parts check renders a code build from the front and compares it with the close-up picture: if every part of the close-up is visible in the build, the kind is built in code, and otherwise it is generated from the close-up with Pixal3D [17] and made solid (Figure 3). Composite objects are split in the inventory, so that every real-world object becomes one model: a console, for example, becomes a desk carrying separate monitors and keyboards.

Gap: the parts check's order. A queued change reverses the order in which the parts check tries the two routes, generation first; this section is rewritten when it lands.

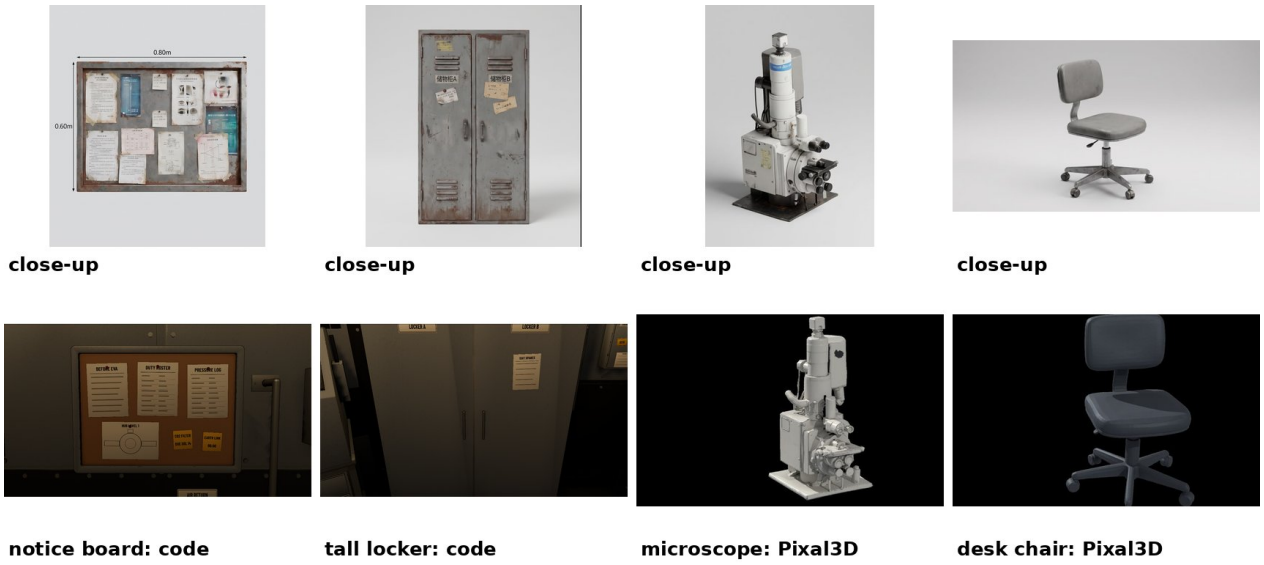


Figure 3: The parts check. Top: the close-ups of four objects. Bottom: each object as built. The notice board and the tall locker are built in code, because their code builds show every part of their close-ups; the microscope and the desk chair are generated with Pixal3D, because no code build shows all of their parts.

2.4 Shape and surface apart



Figure 4: (a) Three library surfaces (painted panel, bare steel and deck plate) at wear settings 0, 0.4 and 0.8. (b) The hub's tool board when each generated piece kept the colours of its own picture (left), and after every surface came from the library and each tool became its own model (right), from the same camera.

Models and code builders give shape only. Every part of every object takes one surface from a shared library of rule-based surfaces, written as ProcFunc functions [32] that are built on Infinigen's shaders [9] and take their colours from the place's palette, with wear only where it has a cause, such as edges (Figure 4a, Appendix F). On a generated model, PartCrafter [21] splits the shape into parts, and an open vision-language model, shown each part outlined on the close-up, names the library surface it takes; the picture's colours are not kept. Figure 4b shows why: when each piece kept the colours of its own picture, every piece brought its own rust and stains, and the room looked inconsistent. This made the hub consistent, and the creator accepted it (Table C1). Sound is attached in the same way, to surfaces, rooms and objects, and is generated by MOSS-SoundEffect [24] (Appendix F).

Gap: painting beyond the hub. On the rest of world 1 the painting is not yet consistent (Appendix B, Limitations); this section is rewritten when the surface stage's new method has been measured there. That method is the finish-map route: Gemini 3.1 Pro [33] lists a model's finishes on its close-up and names a library surface for each, Nano Banana Pro draws the close-up again as a map with one flat colour per finish, and SegviGen [34] lays that map onto the 3D shape.

2.5 Terrain

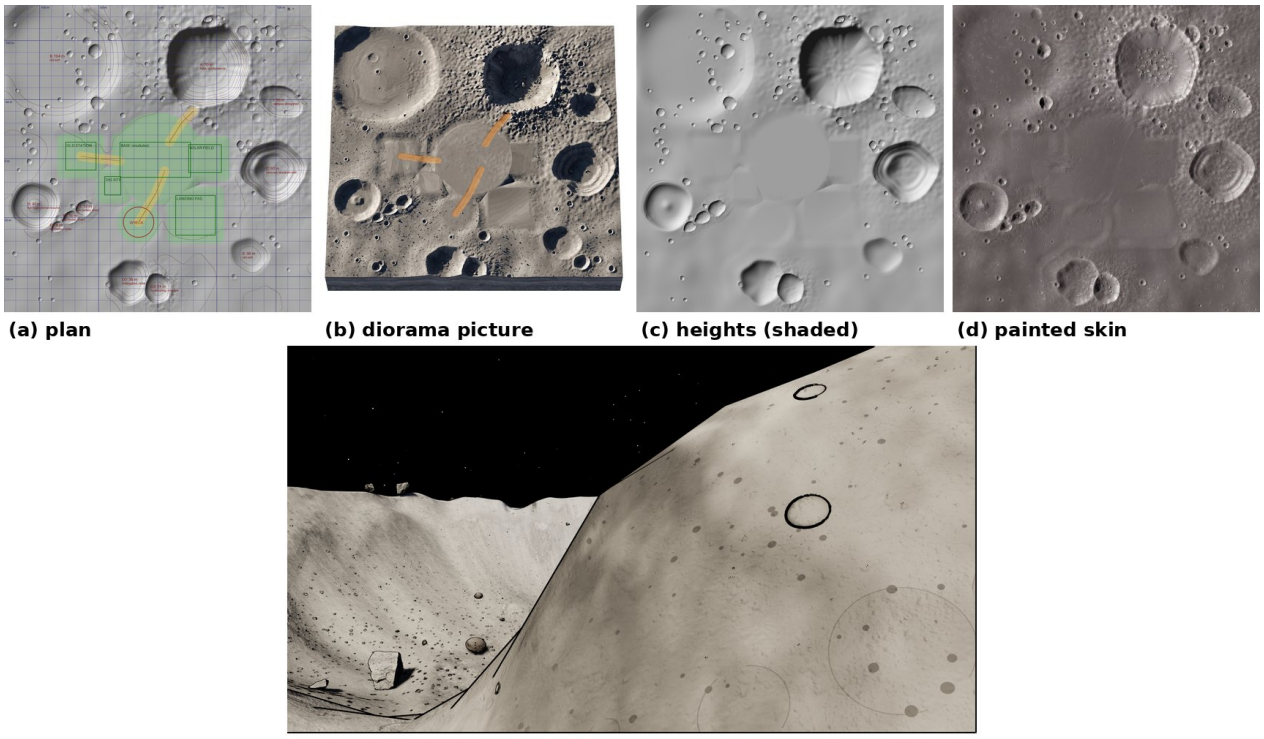


Figure 5: Terrain around the Moon base. (a) The dimensioned plan with flat areas and paths. (b) A diorama picture drawn over the plan. (c) The heights that follow it, shaded. (d) The skin painted over the plan. (e) The result in the engine, from a crater rim.

Large outdoor places follow the same pattern (Figure 5). A dimensioned plan places the flat areas and paths the place needs (a), a diorama picture drawn over the plan sets the landforms (b), and the heights follow the picture (c). A skin is painted over the plan, with fine relief from monocular depth [20] (d), rocks are single generated models, and everything is built on the world's own curved ground (e).

2.6 Checks before spending

Deterministic checks run before every stage that costs money. They test real geometry, because a fault costs nothing to fix in the plan but an evening of the creator's time once the place is built. The checks look for light leaking out of the room, pieces in front of openings or off their surface, doors that do not separate their two sides, models that are open, too thin or leaning, and surfaces off the palette (Appendix D). A room whose shell is laid as a kit of made pieces is scored as a partition of its surfaces, after Point2Part's requirement that parts neither overlap nor leave gaps [35]: every



(a) kit as first built **(b) new route after the placement check**

Figure 6: A fault caught by the placement check. (a) In the first build of the hub, a conduit box covered the porthole. (b) In the current framework, the placement check found the box in front of the opening and moved it 0.5 m along the wall before anything was spent.

visible surface should belong to exactly one piece. A failed check sends the work back one stage (Figure 6).

2.7 The canonical scene

Every place is exported as one OpenUSD stage [36] in which each object carries its kind, its transform in metres, its collision, the library surface of each part and that surface's sounds, with the generated base layer under an edit layer that belongs to the creator, so that an edit survives when the base is generated again (Figure 7). A test place checks this across a base that changed; on a real place it is shown only for a base generated again, not yet for one whose models changed.

A scene record adds what the place needs around its inventory's objects, built by code builders and painted from the library: the room's shell, the ground out to the horizon, water, the far backdrop, the neighbouring places seen through open doorways, every light, and the cameras that the place is judged from.

Before a place is accepted, its loose objects are dropped in a physics simulation on the stage's own ground with their own triangles, and each rest pose is written back into the layout. Settling only corrects: an object that would turn or drift further than a small limit keeps its laid pose and fails the resting check, which also fails anything floating, tipping, sunk into the ground or lying inside another piece (Appendix D). A large move is a fault of the layout, not of physics, and the layout follows the creator's concept.

A place is complete only when its evidence says so. Coding agents judging their own scenes from renders agree with a measured score on geometry only 45.8 percent of the time, below chance [15], and ours called places finished that the creator then found unfinished. SCORE therefore computes completeness instead of asking for it. A completion gate passes a place only when every inventory row has its object on the stage or a written reason for dropping it, nothing visible stands in for a made piece, no real resting fault is left, every model the creator picked is still the one shown, every check was run on the stage as it is now, and the review page was rendered from that stage. Every check answers pass, fail or unknown, and unknown blocks like fail, as in LEGO-Anything's validator; a hook in the agent's harness refuses to end a session that claims a place is done while its gate does not pass. The gate's checks are listed in Appendix D and its counts on world 1 in

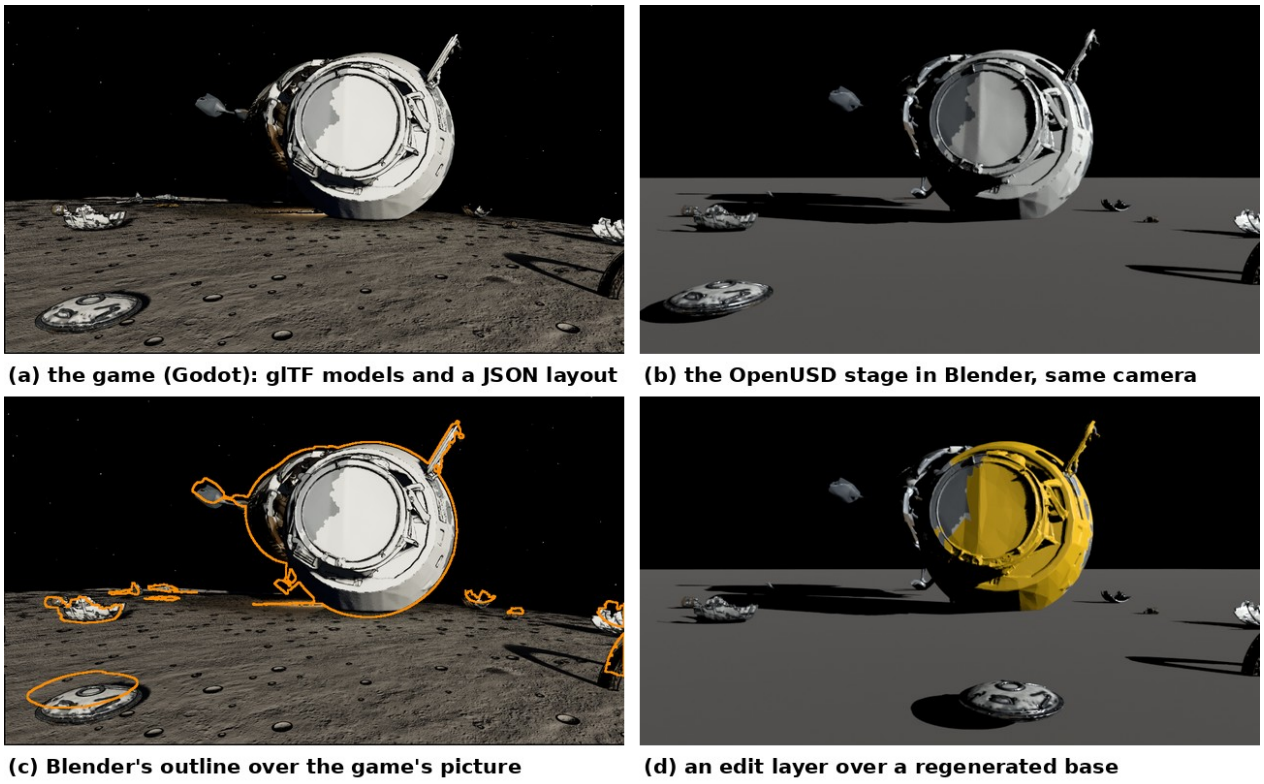


Figure 7: The wreck, an outdoor place of the game 2099, as an OpenUSD stage. (a) The place in the game, from one of its fixed cameras. (b) The stage rendered in Blender from the same camera. (c) Blender's object outlines over the game's picture. (d) The same camera after a scripted edit in the edit layer (the round cover moved, the capsule's hull given another library surface) and after the base layer was generated again.

Appendix B.

The resting check is a triage. Each object is judged by what holds it up, read from its inventory row (the floor, a wall, the ceiling, the object it stands on, a hook it hangs from, or the kit's own frame), with SceneEval's guards against false alarms [37] and SAGE's stability rule for dropped objects [38] (Appendix D). The real faults are grouped by cause, each group with one close-up aimed at its worst case.

The creator's pick of a model is written onto each of its objects as the file's hash, and an export that would show anything else for a picked row is refused unless the creator's decision to replace it is written down. As a candidate check that no gate reads yet, each object is rendered alone and compared with its close-up by its outline and by DINOv2 similarity [16], and the least similar are listed first. The collision queries behind these checks follow the interface of ProcFunc's scene tools [32].

A place's people are part of its stage. Its cast records who is there, where, doing what and why; each person is a skinned character (UsdSkel) in a layer of its own between the creator's edit layer and the base, and a crowd is one point instancer of a cheap body. The framework's character maker turns a short spec, a name and a picture or a few words, into a rigged, clothed, textured and animated character on one rented card: SAM3DBody-cpp [39], a C++ and ONNX runner of SAM 3D Body [22], reads the body off the picture, Kimodo [23] writes the clips from sentences, and GarmentCode's sewing patterns [40] are draped by Newton's cloth solver [41] (Table E1 lists the rest). The drapes, which the creator rejected, and the animals are trials that run only when a spec asks for them (Limitations).

Sound, motion and particle effects are layers of the stage as well. Every sound is a UsdMedia spatial audio prim: room tones loop without a position, an object's hum loops where it stands, and each library surface points at its footstep, impact and scrape sounds, at the levels of the game 2099, now the framework's sound catalogue; nobody has yet listened to this layer in a review. What moves is written as OpenUSD time samples at the game's own speeds (the airlock's doors sink 2.8 m into the floor at 3 m a second), with the sounds the game plays with each motion. Each particle emitter is written both as its parameters, for an engine's own particle system, and as seeded points baked over a loop that repeats exactly, for any USD reader. Following the creator's rule, effects start from real footage and use many fine particles.

The framework stands apart from the game it was first built for. Every file a record names is in the repository or in one release of it, each listed with its checksum, its source and its licence, and a check fails when any code or data names a file of the game's own tree.

Gap: a third home for files. A separate repository of locked, versioned parts that worlds reuse (score_lib) is queued; this paragraph is rewritten when it lands.

Gap: what world 1's stages still lack. Five parts are designed and not yet built: scene records made from each place's plan and concept, the far city's missing kinds of tower, the machines, the greenhouse's crops, and casts on every place whose concept shows people. The paragraphs above on scene records and people are rewritten as they land.

Gap: the scene in a game engine. The same stage loaded in a game engine with its collision, surfaces, and each object's mass and friction. It waits on the game engine adapter and on recording mass and friction.

2.8 Review tools

Each place ends with the creator's review. A review page is a static folder built only from the files the stages wrote: each stage's output side by side, before-and-after shots from the same cameras, the result of every check, and the assembled scene rendered by Blender from the place's OpenUSD stage, from the cameras of its scene record and along a walk through it (Figure 8). Each view is laid beside the game's own shot from the same spot, and a view that is much darker than the game's fails a brightness check, which is how lamps whose light fell off too fast were found. An optional world step can turn the concept into a whole room that can be walked through, so that walls and objects the concept does not show can be seen; nothing it produces is shipped.

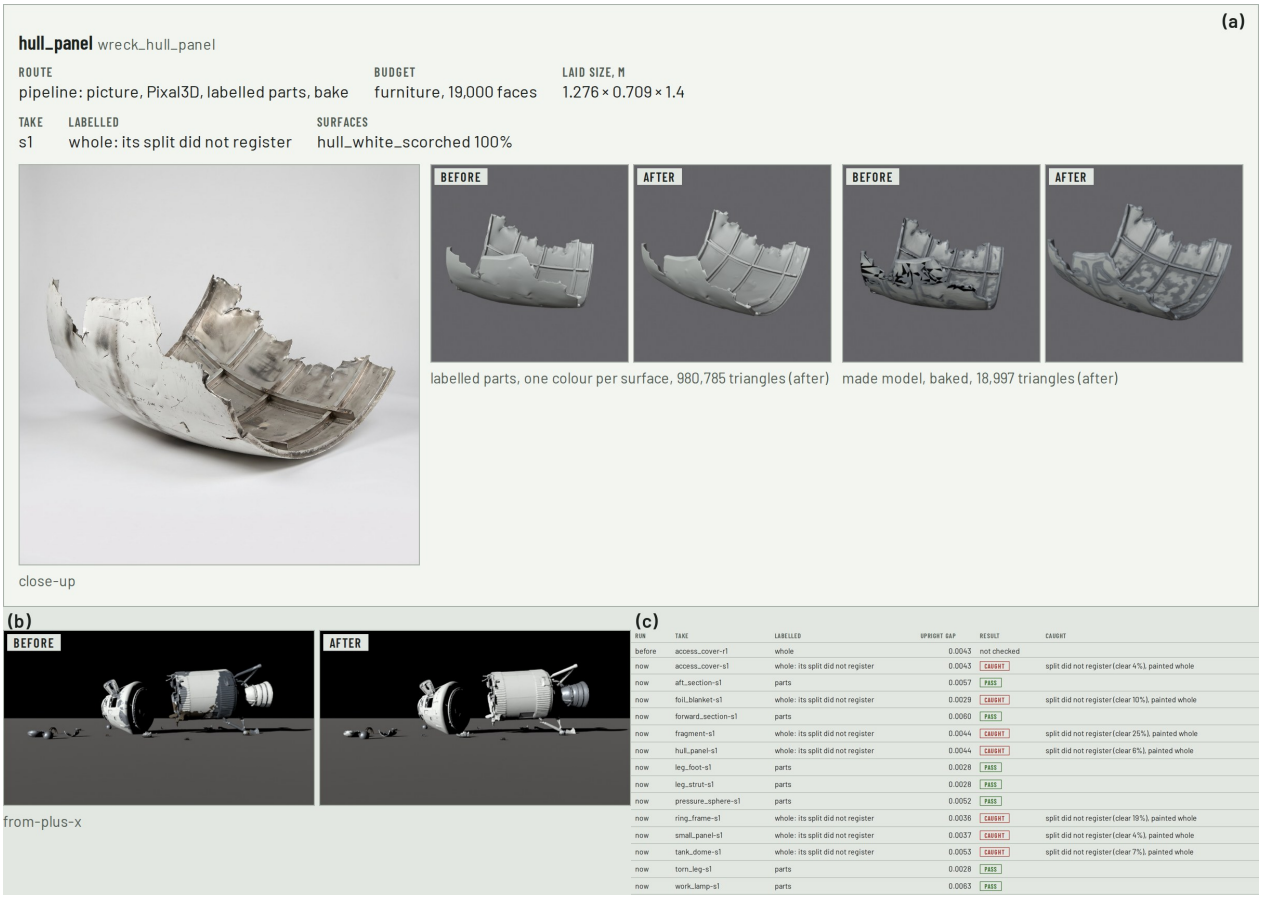


Figure 8: The wreck's review page, from two runs of the route over the same place. (a) One model's close-up, labelled parts (one colour per library surface) and baked model, from the first run and the rerun. (b) The assembled scene from one fixed camera, before and after the rerun. (c) The parts check for each take: in the rerun of 8 October, the split of seven takes did not register against their pictures, so each was painted whole.

3 Results across worlds

We build four worlds with the same stages: the game world of 2099 (a Moon base and its surroundings, a camp on Mars, and a prologue on Earth), a stylised underwater world, a simulation world for robots, and a third-person fantasy world.

Gap: Table 1, results per world and stage. For each world, this table will give the number of places, the wall-clock time, the cost in GPU time and in picture calls, the number of models built in code and generated, and the faults caught before spending at each stage against those found afterwards. It waits until the creator has accepted each stage of each place; world 1's records so far are in Appendix B, and worlds 2 to 4 have not been started.

Gap: figure of one place per world. This figure will show one representative place from each world beside its concept, rendered from the concept's own camera. It waits on the same worlds; every place will be shown in Appendix A.

4 Evaluation

Gap: LEGO-Bench. This section will report SCORE's scores on LEGO-Bench [15], a benchmark of scenes rebuilt from pictures. It waits until the benchmark's code is released and until SCORE's scene export and its Blender adapter are built.

4.1 With and without the agent tools

The tools that check an agent's work (the completion gate, the resting triage with its drop test, the pick lock and the render-and-compare, Section 2) were tested on three places of world 1: a room (the workshop, 814 objects), an outdoor place (the street, 1,198 objects) and a dense room (the hangar, 1,657 objects). In both arms, the same agent (Claude Opus 5.5 in Claude Code, headless) started from the same exported stage and inventory and was asked to make the place as complete as it could in about 50 minutes, through its layout data only; one arm had the framework from before these tools. Each arm ran once, so Table 2 shows what happened, not an average.

Table 2. One session per arm and place. Faults are the resting triage's real faults left on the final stage, measured against the arm's own data; placeholders by the first placeholder check; rows are inventory rows with no object and no written reason. Agent cost is the session's model cost; cloud is its rented machines' cost.

Place, arm	Real faults (start → end)	Placeholders	Rows missing	Minutes	Agent cost	Output tokens	Cloud	Exports	Gate said unknown
Workshop, without	18 → 13	0 → 0	1 → 0	36	$2.26	30.4k	€0.85	7	not used
Workshop, with	18 → 0	0 → 0	1 → 0	26	$1.42	14.3k	€0.85	2	2 of 4 calls
Street, without	11 → 0	0 → 0	0 → 0	38	$2.40	30.0k	€0.32	9	not used
Street, with	12 → 0	0 → 0	0 → 0	39	$1.21	10.4k	€0.76	1	0 of 5 calls
Hangar, without	14 → 4	5 → 0	0 → 0	47	$2.01	26.6k	€0.00	7	not used

Hangar, with	$14 \rightarrow 9$	$5 \rightarrow 0$	$0 \rightarrow 0$	32	\$2.34	29.1k	€0.64	5	3 of 6 calls

The tools made building cheaper more often than they made the scene better. On the workshop and the street, the arm with the tools used about half the agent's output and cost, and exported the stage once or twice instead of seven to nine times, because the triage named the few real faults among hundreds of alarms. On the workshop, 13 of its 18 faults were cleared by writing into the inventory what holds a piece up, which the creator still has to confirm. On the hangar the arm without the tools left fewer faults: the arm with them spent its time replacing five placeholder slabs and trying the drop test, which failed on a room with no ground mesh (fixed after the run), and the nine faults it left are models with stray parts below their feet. No agent claimed a place was complete, so the gate's hook never had to refuse a claim. No row carries an owner's pick, so the pick lock was not tested.

Gap: ablations. The framework with and without the optional world step, and with the checks before spending switched off, on the same places: concept coverage, density, cost, faults found after spending and the creator's review. It waits until every stage runs from the framework repository.

4.2 Style coherence

Gap: style coherence. Whether a place keeps one style. A measure is built (palette fit, lightness spread, blotchiness, and DINOv2 likeness [16] within a place against across places, shape subtracted), and its readings on world 1 are in Appendix B. It becomes a result once it ranks the models the creator faulted below those he accepted, which waits on the surface stage's reviewed before and after.

Gap: robots in the simulation world. A rigged robot loaded with the simulation world in a robotics simulator, with masses, friction and joints written into the scene. It waits on world 3.

5 Limitations

- **One creator, one world so far, and that world not yet accepted.** Every verdict on the results is one creator's, and only the first world has been built. Most of its places do not yet pass their own completion gate (Appendix B), and the creator has accepted one place, the hub.
- **Code builders do not scale well.** They give clean objects, but a coding agent has to write one for every kind of object.
- **Places come out sparser than their concepts.** The concept-density check catches this but does not fix it.
- **Two stages depend on closed, paid models.** Nano Banana Pro draws the concepts and every close-up that the open model's pictures fail; on the last 17 close-ups of world 1, none of the open model's pictures passed the shape check (Appendix E). The surface stage uses two of Google's models: Gemini 3.1 Pro lists each model's finishes on its close-up and names their library surfaces, and Nano Banana Pro draws the finish map that SegviGen lays onto the shape. A public framework thus depends on closed services for two of its stages. Their daily limits set the pace of world 1, and they can change their price or be retired, which limits reproducibility. An open-model fallback for both stages is future work.
- **Parts do not follow finishes, and shadow reads as paint.** PartCrafter's parts follow shape, not paint, so a finish without its own part, such as a gold foil patch on a white hull, is

lost when the part is painted with one surface, and the judge that names each part's surface reads shadow on the close-up as dark paint. A split seeded by the close-up's regions (GeoSAM2 [42]) owns more finishes in a comparison on 22 models, but is not yet part of the route.

- **Objects are physical but not interactive.** Seats, terminals and other usable objects do not work in the output.
- **Characters' clothes are rejected, and animals' motion is rough.** The creator rejected the Newton drapes that every crew and cast body wears, as bloated at the chest and sleeves, and they fail a silhouette check against Warp's drapes; of the open routes tried since, Newton with settings per look passes the work suits but no space suit, and Pixal3D's mesh with SOMA-X's skeleton fitted inside it fails the check that the clothes stay tucked in. The animals' clips are made in code: the dog's paws twist, its sit reads as a crouch and its feet slide by up to 5.4 cm.

6 Future work

- Build the three further worlds: a stylised underwater world, a simulation world for robots, and a third-person fantasy world.
- Add a game engine adapter for the OpenUSD scene, and write each object's mass and friction into it.
- Make assets interactable, which robotics in particular needs.
- Finish the animals and the drapes: animal motion retargeted from a captured set whose licence has been checked, and a drape route that passes the silhouette check on every look and the creator's review.
- Make the surface stage hold on world 1: parts split along finishes, read from close-ups without shadow.
- Make the open picture model's close-ups pass the shape check more often.
- Wrap open solvers as stages for worlds at other scales, starting with orbital dynamics through REBOUND [43].
- Evaluate SCORE on LEGO-Bench [15].

7 Conclusion

SCORE builds complete, owned and responsive worlds from a creator's choices, by joining open vision foundation models, procedural tools and coding agents into fixed stages whose outputs are kept. On its first world, the parts check, one model per real-world object and the shared library of rule-based surfaces made one room, the hub, consistent enough for its creator to accept it, and the checks before every stage that costs money caught faults before money was spent. Only 3 of that world's 18 places passed its completion gate when it was last measured, and the hub was not one of them. Whether the framework holds across a whole world, and across kinds of worlds, is what the remaining places, worlds and the benchmark will show.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[16] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[17] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[18] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[19] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[20] L. Yang, B. Kang, Z. Huang, *et al.*, "Depth Anything V2," *arXiv preprint arXiv:2406.09414*, 2024.

[21] Y. Lin *et al.*, "PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers," in *Advances in neural information processing systems (NeurIPS)*, 2025.

[22] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[23] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[24] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[25] Y. Li *et al.*, "Chorus: Multi-teacher pretraining for holistic 3D Gaussian scene encoding," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2026.

[26] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[27] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[28] B. Lei, Y. Li, X. Liu, *et al.*, "Hunyuan3D Studio: End-to-end AI pipeline for game-ready 3D asset generation," *arXiv preprint arXiv:2509.12815*, 2025.

[29] Qwen Team, "Qwen-Image-Edit-2511 model card." Hugging Face, 2025. Available: https://huggingface.co/Qwen/Qwen-Image-Edit-2511

[30] Qwen Team, "Qwen3.8-27B model card." Hugging Face, 2026. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[31] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[32] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[33] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[34] L. Li *et al.*, "SegviGen: Repurposing 3D generative model for part segmentation," *arXiv preprint arXiv:2603.16869*, 2026.

[35] H.-T. Tsui, Y.-R. Tuan, X. Ma, N. Ugrinovic, T. Shiratori, and K. Kitani, "Point2Part: Unified 3D partitioning from point prompts," *arXiv preprint arXiv:2609.38180*, 2026.

[36] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[37] H. I. I. Tam, H. I. D. Pun, A. T. Wang, A. X. Chang, and M. Savva, "SceneEval: Evaluating semantic coherence in text-conditioned 3D indoor scene synthesis," *arXiv preprint arXiv:2503.14756*, 2025.

[38] H. Xia *et al.*, "SAGE: Scalable agentic 3D scene generation for embodied AI," *arXiv preprint arXiv:2602.10116*, 2026.

[39] A. Qammaz, "SAM3DBody-cpp: A standalone C++ inference engine for SAM 3D Body." GitHub, 2026. Available: https://github.com/AmmarkoV/SAM3DBody-cpp

[40] M. Korosteleva and O. Sorkine-Hornung, "GarmentCode: Programming parametric sewing patterns," *ACM Transactions on Graphics*, vol. 42, no. 6, p. 197, 2023.

[41] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[42] K. Deng *et al.*, "GeoSAM2: Unleashing the power of SAM2 for 3D part segmentation," *arXiv preprint arXiv:2508.14036*, 2025.

[43] H. Rein and S.-F. Liu, "REBOUND: An open-source multi-purpose N-body code for collisional dynamics," *Astronomy & Astrophysics*, vol. 537, p. A128, 2012.

[44] Blender Foundation, "Blender." 2026. Available: https://www.blender.org/

[45] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, "DINOv3," *arXiv preprint arXiv:2508.10104*, 2025.

[46] World Labs, "Terms of Service." 2026. Available: https://docs.worldlabs.ai/terms-of-service

[47] Black Forest Labs, "FLUX.2 [klein] 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-4B

[48] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large language models are secretly powerful text encoders," in *Conference on language modeling (COLM)*, 2024.

[49] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[50] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[51] C. Ye *et al.*, "Hi3DGen: High-fidelity 3D geometry generation from images via normal bridging," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2025.

[52] Black Forest Labs, "FLUX.2 [klein] Base 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B

[53] thedeoxen, "refcontrol: Reference-depth LoRA for FLUX.2 [klein] 4B." Hugging Face, 2026. Available: https://huggingface.co/thedeoxen/refcontrol-FLUX.2-klein-4B-reference-depth-lora

[54] J.-P. Zhang, C.-F. Pu, M.-H. Guo, Y.-P. Cao, and S.-M. Hu, "One model to rig them all: Diverse skeleton rigging with UniRig," *ACM Transactions on Graphics (SIGGRAPH)*, 2025.

[55] R. Abdrashitov, K. Raichstat, J. Monsen, and D. Hill, "Robust skin weights transfer via weight inpainting," in *SIGGRAPH asia 2023 technical communications*, 2023.

[56] Anthropic, "Claude Code." 2026. Available: https://www.anthropic.com/claude-code

Appendix

A Per-world demos

A.1 World 1: the game world of 2099

Gap: world 1's demo. One frame and a walk-through of every place of world 1, rendered from the stage the creator accepted for it. A first render of 9 October (5,292 frames on 20 rented L4 cards, 78 minutes, €15.82) is not shown, because its stages have changed since.

Figure A1 follows two objects through the stages, from the concept to the place.

Gap: worlds 2 to 4, and a public link to the videos. This appendix will show every place of the other three worlds in the same way. The videos of world 1 are not yet published at a public address.

B Detailed per-stage results

Table B1 gives the records of world 1's 17 places as the build logs and the cloud ledger hold them. They are records, not yet results: the creator has accepted only the hub, the old station and the wreck have changed since he reviewed them, and no other place has been reviewed. The cloud costs are lower bounds, because batches such as review renders and the shape-check judge could not be attributed to one place, and no cost is recorded per stage of a place, only per batch. The resting check is the one on each place's review page of 8 October; it counts every piece it fails, including many deck plates that it finds sunk by 4 cm, not yet judged as real faults or not.

Table B1. World 1, per place. Models are those built in code and those generated. Cloud is the



Figure 9: Two objects through the stages: the lab's desk chair (top) and the wreck's forward section (bottom). From left: the concept the creator chose, the close-up drawn from it, the generated model with its parts, the model painted from the surface library, and the model in its place on the stage. The forward section shows a known fault: its scorched paint has no part of its own and is lost in painting (Section 5).

ledger's rented machine time attributed to the place, in euros; pictures are the picture model's calls in dollars, from the build logs. Faults are those the checks caught before money was spent and those found after.

Place	Rows	Pieces	Models, code / generated	Cloud	Pictures	Faults before / after	Resting
Hub	74	460	88 / 12	€3.65	$1.07	not counted	405 of 460
Lab	47	412	54 / 22	€8.36	$4.15	13 / 12	373 of 412
Greenhouse	59	1,248	146 / 3	€3.07	$2.13	7 / 3	1,154 of 1,248
Workshop	54	814	97 / 8	€2.18	$3.35	3 / 9	671 of 814
Airlock	33	330	73 / 1	€2.53	$0.13	1 / 5	296 of 330
Walkway tube	13	39	23 / 0	€0.81	$0	0 / 2	38 of 39
Habitat	53	637	100 / 2	€5.70	$1.34	1 / 4	543 of 637
Garage	41	602	75 / 19	€4.29	$8.71 with the hangar	3 / not counted, with the hangar	493 of 602
Hangar	37	1,686	61 / 13	€4.22	with the garage	with the garage	865 of 1,686
Wreck	14	22	3 / 13	€4.88	$2.68	2 / 1	22 of 22
Old station	17	28	10 / 14	€4.93	$3.22	5 / 2	29 of 29
Mars camp	33	218	51 / 5	€3.47	$1.21	4 / 8	127 of 218

Place	Rows	Pieces	Models, code / generated	Cloud	Pictures	Faults before / after	Resting
Launch view	4	67	4 / 1	€0.33	$0.40	1 / 2	8 of 67
Square	40	4,135	74 / 2	€1.64	$1.07	3 / 4	3,712 of 4,135
Street	33	1,198	45 / 2	€0.98	$1.74	7 / 6	1,179 of 1,198
Stairwell	21	118	42 / 1	€1.23	$2.41 with the flat	2 / 6	117 of 118
Flat	38	52	35 / 9	€2.44	with the stairwell	5 / 6	50 of 52
World 1	611	12,066	981 / 127	€54.71			10,082 of 12,066

B.1 World 1's checks, as last measured

- **Settling.** On the wreck's stage of 8 October, 11 of 22 pieces rested as first laid; once the coordinating agent laid the wreck out again after its concept and cut shallow dents where pieces had struck the ground, all 22 rested. On the old station's stage, the check found a pile of sandbags lying inside the lander's legs, and once the pile was moved, all 29 pieces rested. Both places have changed since the creator reviewed them, and on 10 October both failed the completion gate.
- **Completion gate.** On 10 October, with the gate of that day, which still counted the reach check and used the first placeholder check, 3 of the 18 places passed, 14 failed and one was unknown; the gate as Section 2 describes it has not yet been measured over the whole world. The first placeholder check judged only plain primitives, so the hangar's ship, which a code builder assembled from a hull and legs, passed it until the creator saw it as a placeholder in a demo of the world.
- **Resting triage.** Over world 1's 17 places, the plain check flagged 4,078 of 12,032 objects; the triage kept 104 objects with real faults in 85 groups and marked the other 3,974 as false alarms, most of them kit pieces that stand where the kit lays them. The drop test did not finish on the square in three hours. These counts measure detection, not repair, and were taken with the drop test of 8 and 9 October, whose two faults were corrected on 10 October; they have not been measured again.
- **Render-and-compare.** In the lab, the four least similar objects were a mouse drawn black, a ceiling lamp without its lens, a scanner without its screen and a wall monitor with a blank screen.
- **Painting.** 41 of world 1's 129 stored takes still carry an early colour pick per PartCrafter part and one more was painted per part by the judge without regions; later takes are split along SAM 2.1 regions of the close-up before the judge names each surface, and that method still fails four of six named models, because one part can hold two finishes and the judge reads shadow as dark paint.
- **Characters.** Rebuilt from her own picture, Nev, a crew member of the Moon base, took 11.2 minutes of an H100, and a new person made from one sentence took 21 minutes.

B.2 Style readings

Table B2 gives the style readings for every place of world 1 with more than two generated models, as a record, not a result (Section 4). The lightness spread is the standard deviation of the models' median lightness (L*); over all 140 models it is 15.0. Marks is the median over the models of the 75th percentile of the L* spread within tiles of 8 by 8 pixels. Likeness is the mean DINOv2 class-token cosine of pairs within the place and across places, for the renders and for their silhouettes, and the gap is the renders' difference less the silhouettes', with any two renders of one model left out.

Table B2. Style readings of world 1's generated models, per place.

Place	Models	Lightness spread	Marks	Likeness within / across	Silhouettes within / across	Gap
Camp	5	8.3	4.55	0.345 / 0.136	0.378 / 0.164	−0.005
Flat	9	16.7	4.83	0.186 / 0.122	0.243 / 0.176	−0.003
Garage	19	12.6	2.80	0.185 / 0.159	0.199 / 0.177	0.004
Greenhouse	3	4.5	6.12	0.071 / 0.140	0.140 / 0.174	−0.035
Hangar	13	9.5	2.99	0.185 / 0.151	0.231 / 0.174	−0.023
Hub	12	3.8	5.96	0.243 / 0.151	0.243 / 0.160	0.009
Lab	22	14.4	1.96	0.203 / 0.162	0.195 / 0.174	0.020
Old station	24	12.7	8.96	0.221 / 0.156	0.217 / 0.175	0.023
Workshop	8	3.8	7.28	0.402 / 0.158	0.400 / 0.178	0.022
Wreck	16	12.8	3.98	0.241 / 0.161	0.257 / 0.175	−0.002

C Development history

The framework grew out of building the game 2099 between late September and early October 2026, with one owner directing coding agents. In its first form, rooms were laid out in code, a world model painted depth panoramas of them as a scene plan, props came from a picture-to-3D pipeline, and coding agents fitted each room to its plan. Eight places were fitted out in a single day, with layouts close to their plans but mixed styles, because every pass added things to a room and no pass checked what had been placed against the plan. This led to the inventory as a strict list and to deciding the look before any money is spent. Each room's structure then came from concept cutaways drawn from the owner's reference pictures (for the hub he picked one of twenty), and scale moved into a dimensioned plan drawn before the concept, after the first cutaway came out about 13 m across, judged by the people drawn in it, against the 8 to 9 m asked for.

C.1 The hub in six rounds

The mechanisms of Section 2 then came out of six rounds of work on the hub on 6 and 7 October 2026, each answering the creator's verdict on the round before (Table C1). In the first two rounds, he chose between the old and the new method side by side from the same cameras.

Table C1. The hub's six rounds. Cost is given as rented GPU time in euros and picture-model calls in dollars. Frame time is the mean frame time on an RTX 5080, in the night scene, at 1600×900.

Round	Change	Creator's verdict	Cost	Frame time
1, two walls, A/B	surface library, routing by shape, room checks	chose the new method ("Y looks much cleaner")	€0.88	7.81 ms vs 7.53
2, two walls, A/B	71-variant library, shared texture sets, roof check	chose the new method again	€1.70	8.05 ms vs 8.24
3, whole room	all stages end to end, 74 models	old models were still loaded; labels and floor fittings were wrong	€1.67	6.78 ms (old 9.22)
4, whole room	detail through the pipeline, picture colours kept	"pretty much all of this became messy"	€8.50, $5.09	9.6 ms
5, eight pieces, A/B	shape only from the pipeline, library surfaces, parts check	new method preferred for the door, notice board and porthole; three pieces still wrong	€0.21	not run
6, whole room	round 5 everywhere, one model per object, straightness check	"this looks amazing, I have no negative feedback"	€3.48, $1.07	5.6 ms

Three findings shaped the method. First, deterministic checks catch what agents miss: in round one, the share of rays leaking out through walls and roof fell from 1.53 percent of 5,817 to zero, and the colour noise within surfaces from a median of 31.1 to 3.2 percent, but only walking through the loaded room showed 26 old models still in it, which became the made-only check. Second, generated pieces that kept their pictures' colours made the room inconsistent, so surfaces come only from the library. Third, generated composite objects melted their parts together, and generated boxes leaned (one locker by 13 degrees), which led to one model per real-world object and to the straightness check (Figure 4b). By round six the room used 0.45 million triangles, against 1.96 million in round four, and 187 MB on disk, against 487 MB.

D Check details

- **Room checks** take about two seconds per room on the laid-out meshes. They cast rays to find light leaking out through walls and roof, find pieces in front of a real opening and roof and wall fittings more than 3 cm off their surface or turned more than 5 degrees from it, and count pieces, triangles and lamps against the room's budget.
- **Door check:** every door must stand in a wall or partition that fully separates its two sides, tested by walking a grid from one side to the other.
- **Parts check:** in the code build, every part shown in the close-up must be at least 10 percent visible from the front, and no label may cover a part.
- **Model check:** every model must be a closed solid with walls at least 3 mm thick. A generated box fails if one of its sides tilts more than 2 degrees or is warped. Making a generated model solid stops if its bounding box grows more than 3 percent.
- **Palette sweep and storage budget:** every model's colours are compared with the place's library palette, and each room may use at most 250 MB on disk and 640 MB of textures on the graphics card.
- **Scene check and made-only check:** in the engine, these find anything that floats, sinks or overlaps, and any visible mesh that the framework did not make. A made-only finding can never be waived.
- **Close-up shape check:** the picture needs a plain border all round, one object filling a sensible share of the frame, and an outline within a set proportion of the inventory row's box. The judge's seven questions (among them one object, a clean background, the whole object, the same object, no parts missing or added, the right proportions) each pass by a majority of three answers sampled with different seeds. A close-up passes only when both halves pass.
- **Settling and resting check:** an object that would turn more than 15 degrees or drift more than 30 cm when dropped keeps its laid pose and is marked as not resting; small debris, with no side over 1.4 m, may turn freely and drift up to 1 m. Hung objects are exempt by their inventory anchor, and fixed objects are not judged for tipping.
- **Resting triage:** a glowing part or screen is judged with its host; a fixed row stands as laid but may not float unless joined to another piece; a kit's shell pieces stand in the kit's frame; a hanging, wall or ceiling row needs one contact with anything; a floor row and a row standing on another need four points of their footing (the points within 10 cm of their lowest) within 3 cm of a surface under them, with their weight within 5 cm of the area those points span. An overlap deeper than 3 cm counts only if the two still meet after the loose one moves 5 mm away. Cables may sink 6 cm, cloths 8 cm and ground debris 30 cm. From a drop run, a loose object that moves more than 0.2 m or turns more than 8 degrees does not rest as laid, after SAGE's stability rule, and is a real fault.
- **Completion gate:** the requirements given in Section 2 under the canonical scene, each with its result tied to a fingerprint of the stage's files and inputs; a result made on another stage counts as unknown. The placeholder check judges every mesh a code builder draws, and only the shell builders pass unnamed. A reach check, in which a walker on the place's floor must reach every row, runs beside the gate but is not counted, because it fails backdrop rows that nobody walks to.
- **Render-and-compare:** each object with a close-up is rendered from eight directions 20 degrees up, and the direction with the highest DINOv2 likeness stands for the close-up's camera; the outline score is the overlap of the two silhouettes, each cut to its box.
- **Brightness check:** a review view whose mean brightness is under half that of the game's shot from the same spot fails.

E Models and licences

Every model in the framework must allow commercial use of its output (Table E1); we read each licence for its clauses on commercial use. Candidates were dropped under this rule, among them NVIDIA's Lyra 2.0 (weights for internal research only), Hunyuan3D 2.1 (licence does not apply in the European Union) and GarmentCode's fork of NVIDIA Warp (non-commercial research only), which Newton's cloth solver on upstream Warp replaced; Blender's cloth simulation [44] is kept as a second route. The one open doubt is the face LoRA, whose training data is not disclosed.

Table E1. Models in the framework.

Model	Role	Licence
Nano Banana Pro [31]	concepts, close-ups the open model fails, ground skins	paid API
Nano Banana Pro [31]	finish maps: the close-up drawn again with one flat colour per finish	paid API
Gemini 3.1 Pro [33]	each model's finishes listed on its close-up, and their library surfaces named	paid API
Qwen-Image-Edit-2511 [29]	close-ups, first	Apache-2.0
Qwen3.8-27B [30]	the close-ups' shape check; each part's material	Apache-2.0
Pixal3D [17]	picture to 3D	MIT; DINOv3 licence [45] for its encoder
PartCrafter [21]	parts	MIT; its non-commercial background remover not used
SegviGen [34]	lays the close-up's finish map onto the 3D shape (surface stage)	MIT, on TRELLIS.2 (MIT); DINOv3 licence for its encoder; its non-commercial background remover and nvdiffrast not used
GeoSAM2 [42]	parts of our own model, seeded by the close-up's regions (trial)	Apache-2.0
Depth Anything V2 Small [20]	ground relief	Apache-2.0
ProcFunc, Infinigen shaders	surfaces	BSD-3-Clause
MOSS-SoundEffect v2.0, CLAP	sound	Apache-2.0
World Labs [46]	optional world step	paid; outputs owned by paid users
FLUX.2 klein 4B [47]	earlier prop pictures; the characters' A-pose pictures from words and their hair's clay pictures	Apache-2.0
MoGe-2 [19], SAM 3 [18]	measuring and finding objects in pictures	MIT; SAM Licence

Model	Role	Licence
Kimodo [23]	the characters' motion from sentences	Apache-2.0 code; NVIDIA Open Model License for the Kimodo-SOMA-RP-v1.1 weights, commercial use allowed
LLM2Vec [48] on Meta-Llama-3-8B-Instruct [49]	Kimodo's text encoder, used only while clips are made; nothing from it ships	Llama 3 Community License: commercial use allowed, the 700 million monthly users clause does not apply, "Built with Meta Llama 3" shown, no exclusion in the European Union for the text-only 8B model
SAM 3D Body [22], SAM3DBody-cpp [39]	body shape from a picture	SAM Licence for the weights, SAM3DBody-cpp's converted ONNX and GGUF weights included whatever their tag says; MIT for SAM3DBody-cpp's code
SOMA-X and MHR	the characters' bodies	Apache-2.0
GarmentCode [40], Newton [41] on Warp [50], Blender's cloth [44]	the clothes' sewing patterns and their drape	MIT; Newton and Warp Apache-2.0; Blender is GPL and its output is ours; GarmentCode's Warp fork (non-commercial research only) not used
Hi3DGen [51]	the characters' hair	MIT code; MIT and Apache-2.0 weights
FLUX.2 klein base 4B [52] with the refcontrol reference-depth LoRA [53]	the characters' faces	Apache-2.0, both; the LoRA's training data is not disclosed
UniRig [54]	the skeleton and skin of a quadruped	MIT code and weights
Robust Skin Weights Transfer [55]	the SOMA-X skeleton's skin weights moved onto a Pixal3D person (a trial)	MIT reference code, ported with trimesh and SciPy; libigl, whose bindings carry GPL-3.0, not used
MakeHuman eyes	the characters' eyes	CC0

On our tier, Nano Banana Pro allows 250 pictures a day and 20 a minute; the cheaper Nano Banana (gemini-2.5-flash-image) allows 2,000 a day and 500 a minute. Given the same 10 lab objects, prompts and inputs, the cheaper model refused the standard prompt for 6 and drew those only from the crop alone; of its pictures, 2 were good, 3 usable, 1 marginal and 3 the wrong shape, 1 object was refused, and the objects came out small in frames of about one megapixel. Close-ups therefore stay on Nano Banana Pro.

The close-ups' shape check was tuned on 40 close-ups that four open picture models drew of the lab's ten objects, each scored by hand, and on Nano Banana Pro's ten pictures of the same objects. With the majority of three sampled answers, the check agreed with the hand score on 29 of the 40, passed all ten of the closed model's pictures and two marginal ones, passed none scored wrong or failed, and failed nine that were good or usable. The judge's single answer at temperature zero was rejected: asked the same 51 questions twice in one batch, it changed its verdict on 7. On the lab, 7 of the closed model's first pictures failed for drawing the room or dimension lines around the object, and all 7 passed when redrawn from the crop alone. Table E2 gives the stage's runs on world 1.

Table E2. The close-up stage on world 1. Cloud is rented card time in euros; the closed model's pictures are in dollars.

Run	Close-ups	Open model accepted	Closed model accepted	Left for the creator	Cloud	Closed model
Lab, 22 rows (earlier check: one answer)	22	8	14	0	€4.12	$2.81
World 1's remaining rows (three places)	17	0	10	7	€3.06	$4.02

The picture-to-3D step, the part splitting, the surface bakes, the sound generation, all checks and the assembly run automatically. Coding agents (Claude Code [56]) write the plans, the inventories and every code builder; in the hub's final round, 86 of its 98 models were built in code. Some steps were still done by hand: some generated objects were turned to face the right way by eye, and a few labels were placed by hand before a rule took over that job. The concepts and the verdicts are the creator's own choices.

F Surfaces, sound and light

The surface library holds families of variants: painted, steel, aluminium, rubber, plastic, cable, fabric, composite, glass, screen, light and print. After the hub's second round it held 71 variants made from 14 ProcFunc recipes. Each variant is a recipe with settings, takes its colours only from the palette, and is baked into base-colour, metal-roughness and normal maps at a texture density that depends on how close the player can get. Wear has three levels and comes from causes: edges, and kicks within 32 cm of the floor that a piece stands on. Every sound starts from a short written brief about its object. Four takes are generated for each sound, scored by how well CLAP matches them to their prompt minus any measured faults (clipping, unsteady loops or bad joins), and levelled to one loudness per kind of sound under a true peak of -3 dBTP. A creator may swap any take. For the game 2099, all 61 sounds were generated as 244 takes in one batch for €1.61, and the creator chose to keep earlier recordings for three kinds of sound: breathing, a sliding door, and footsteps on steel.

Some surfaces need more than a flat library colour, so the framework writes its own shaders as UsdPreviewSurface materials over pictures it bakes, which any USD reader can draw. The Earth in the Moon's sky, for one, is the game's own noise shader ported to numpy and baked once into a longitude-latitude picture, lit by the place's own sun.

Light adds up, so it is planned as layers. For each kind of module, the bounce light of each group of lamps, and of a few sun directions through the windows, will be baked on rented machines as a separate layer, and the engine will mix these layers live according to each lamp's brightness. Outdoor light and the light on moving objects stay live. This part is not built yet.

G Cost and infrastructure

Every runner records each machine it rents in a cloud ledger. Table G1 gives the ledger's cost and time per kind of batch for October 2026, up to the provider's bill read on 10 October at 01:36 UTC. The ledger holds €547.31 of the bill's €627.74; of the €80.43 between them, €51.87 is not compute (disks, addresses, the image registry, storage), and €26.01 is processor machines that the ledger priced by the minute while the provider bills them by the started hour. Of the ledger, €112.41 was research and development on the framework itself, and €434.90 was the route's batches on world 1, reruns and rejected rounds included; that is not the cost of producing world 1, because no place beyond the hub has been produced end to end and accepted.

Table G1. Cost and time per kind of batch, 1 to 10 October 2026. The column "with the rest of the bill" adds the €80.43 that the ledger does not hold in proportion to each kind's share, an allocation, not a measurement. The median batch is the median wall time of the kind's batches. Engine runs are the game engine's tests, shots and benchmarks; the open picture models are FLUX.2 klein and Qwen-Image-Edit. Processor machines are named by their cores.

Kind of batch	Batches	Machine time	Ledger	Share	With the rest of the bill	Median batch	Main card types
Blender	299	140.9 h	€155.42	28.4%	€178.26	12 min	L4 52%, 16-core 14%
Pixal3D	83	154.2 h	€130.98	23.9%	€150.23	40 min	L4 93%, H100 PCIe 3%
Open judge	82	16.1 h	€55.75	10.2%	€63.95	11 min	H100 PCIe 55%, two H100 SXM 37%
Library bake	238	50.7 h	€46.30	8.5%	€53.11	8 min	L4 84%, 16-core 8%
Characters	16	35.3 h	€35.31	6.5%	€40.50	45 min	L4 72%, H100 PCIe 28%
Parts and segments	119	26.3 h	€33.27	6.1%	€38.16	8 min	L4 53%, H100 PCIe 35%
Engine runs	142	39.3 h	€32.63	6.0%	€37.42	9 min	L4 70%, 32-core 64 GB 30%
Images and proofs	48	10.8 h	€24.06	4.4%	€27.60	10 min	16-core 44%, H100 PCIe 38%
Open picture models	63	6.0 h	€15.96	2.9%	€18.31	3 min	two H100 SXM 46%, H100 PCIe 38%

Kind of batch	Batches	Machine time	Ledger	Share	With the rest of the bill	Median batch	Main card types
Method experiments	32	14.2 h	€15.74	2.9%	€18.05	33 min	L4 35%, 32-core 128 GB 28%
Sound	5	2.1 h	€1.88	0.3%	€2.16	5 min	L4 86%, H100 PCIe 10%
All	1,127	495.9 h	€547.31	100%	€627.74		